CS423 – CSC15003 – Software Testing (AI-augmented · 2026)

HOMEWORK – AI-FIRST EDITION (2026 v1.0)

HW02 – Domain Testing on EShop

Student information
Lê Gia Bảo
23127325

I. Feature: Pool A - FR-03: Forgot password and password reset (two steps)

1.Requirement Analsys

Functional requirements summary

FR-03 defines a two-step password reset flow:

- Step 1: The user enters an already registered email address.
- The system generates a 6-digit OTP and presents it in the demo environment.
- Step 2: The user enters the OTP, a new password, and a confirmation of the new password.
- The new password must follow the same strength rules as FR-01.
- The two password fields must match.
- The OTP must be valid for the same email that initiated the reset.

User inputs

The main user inputs for this feature are:

- Email address (Step 1)
- OTP (Step 2)
- New password (Step 2)
- Confirm new password (Step 2)

Assumptions

- The new password uses the same strength rules defined in FR-01: minimum 8 characters, at least one uppercase, one lowercase, one digit, and one special character.
- OTP is a numeric 6-digit value, represented as a string in the API.
- OTP validity is tied to the specific email that requested the reset.
- The spec does not explicitly define an expiry window or one-time-use rule, but SEC-07 suggests these should be treated as expected constraints.

2. Input Domain Identification

Input 1: Email address

- Data type: String
- Valid domain: A syntactically valid email address that belongs to an existing registered user
- Invalid domain: Empty, malformed, unregistered, non-string values
- Constraints:
    - Must be present
    - Must follow email format
    - Must correspond to a registered account
- Dependencies:
    - Existing user database
    - Reset request initiated for that same email
- Assumptions:
    - The UI uses email input semantics and the backend validates format

Input 2: OTP

- Data type: String
- Valid domain: A 6-digit numeric code that matches the OTP generated for the current reset request
- Invalid domain: Wrong value, wrong length, non-numeric, expired/used, empty
- Constraints:
    - Must be exactly 6 digits
    - Must be associated with the current email reset request
- Dependencies:
    - The email that initiated reset
    - OTP generation record/session state
- Assumptions:
    - OTP is single-use and time-bounded, as implied by security requirements

Input 3: New password

- Data type: String
- Valid domain: A password that meets the strength rules and is not empty
- Invalid domain: Too short, missing required character classes, empty/whitespace, non-string
- Constraints:
    - Minimum 8 characters
    - At least one uppercase letter
    - At least one lowercase letter
    - At least one digit
    - At least one special character from the allowed set
- Dependencies:
    - Password confirmation field
    - Reset request context
- Assumptions:
    - Same rules from FR-01 apply

Input 4: Confirm new password

- Data type: String

- Valid domain: A value identical to the new password
- Invalid domain: Empty, different from the new password, non-string
- Constraints:
  - Must match the new password exactly
- Dependencies:
  - New password input
- Assumptions:
  - Confirmation is only a matching check, not an independent strength check

3. Equivalence Class Partitioning

Email address

- EC1: Valid registered email, e.g. user@domain.com
- EC2: Valid format but unregistered email
- EC3: Empty or whitespace-only input
- EC4: Malformed email format, such as missing @, missing domain, invalid characters
- EC5: Wrong data type, such as null or numeric input

Why these classes exist:

- They separate successful flows from different failure categories.
- They distinguish syntax problems from account-state problems.

OTP

- EC1: Correct 6-digit OTP for the current reset request
- EC2: Incorrect 6-digit OTP for the same email
- EC3: OTP with fewer than 6 digits
- EC4: OTP with more than 6 digits
- EC5: Non-numeric OTP
- EC6: Empty OTP
- EC7: OTP that is expired or already consumed

Why these classes exist:

- They cover normal valid input and all meaningful invalid input patterns related to length, format, and validity state.

New password

- EC1: Strong password meeting all rules
- EC2: Password shorter than 8 characters
- EC3: Password missing uppercase letter
- EC4: Password missing lowercase letter
- EC5: Password missing digit
- EC6: Password missing special character
- EC7: Empty or whitespace-only password

- EC8: Non-string input [redacted]

Why these classes exist: [redacted]

- Each class isolates a single rule violation, which is important for domain-driven validation. [redacted]

Confirm new password [redacted]

- EC1: Matches the new password exactly [redacted]
- EC2: Empty or whitespace-only confirmation [redacted]
- EC3: Different from the new password [redacted]
- EC4: Non-string input [redacted]

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type (Valid / Invalid)
TC-FP-001	Step 1: Submit a registered email to request OTP	Email address	user@domain.com	EC1	System accepts the email, generates a 6-digit OTP, and moves to Step 2	Valid
TC-FP-002	Step 1: Submit a syntactically valid but unregistered email	Email address	unknownuser@domain.com	EC2	System rejects the request with an appropriate error message and does not continue the reset flow	Invalid
TC-FP-003	Step 1: Submit an empty email field	Email address	""	EC3	System rejects the request and prompts for an email	Invalid
TC-FP-004	Step 1: Submit whitespace-only email	Email address	" "	EC3	System rejects the input as invalid	Invalid
TC-FP-005	Step 1: Submit malformed	Email address	invalid-email	EC4	System rejects the input with a	Invalid

	email format				format validation error	
TC-FP-006	Step 1: Submit null email value	Email address	null	EC5	System rejects the request and shows a validation error	Invalid
TC-FP-007	Step 1: Submit non-string email input	Email address	12345	EC5	System rejects the input and does not proceed	Invalid
TC-FP-008	Step 1: Submit a valid email with special characters in the local part or domain	Email address	user+tag@sub.domain.com	EC1	System accepts the email if it is syntactically valid and linked to a registered account	Valid
TC-FP-009	Step 2: Submit valid OTP and matching password fields	OTP, New password, Confirm new password	OTP: 123456; New password: StrongPass1!; Confirm: StrongPass1!	OTP EC1, Password EC1, Confirm EC1	Password reset succeeds and the user is notified of success	Valid
TC-FP-010	Step 2: Submit an incorrect OTP	OTP	999999	OTP EC2	System rejects the reset and shows an OTP validation error	Invalid
TC-FP-011	Step 2: Submit OTP shorter than 6 digits	OTP	12345	OTP EC3	System rejects the OTP as too short	Invalid
TC-FP-012	Step 2: Submit OTP longer than 6 digits	OTP	1234567	OTP EC4	System rejects the OTP as too long	Invalid
TC-FP-013	Step 2: Submit non-numeric OTP	OTP	ABC123	OTP EC5	System rejects the OTP as invalid format	Invalid

TC-FP-014	Step 2: Submit empty OTP	OTP	""	OTP EC6	System rejects the submission and requires OTP entry	Invalid
TC-FP-015	Step 2: Submit an expired or already used OTP	OTP	Previously expired/consumed OTP	OTP EC7	System rejects the reset and states that the OTP is no longer valid	Invalid
TC-FP-016	Step 2: Submit OTP that belongs to a different email context	OTP + email context	OTP generated for another email address	Dependency validation	System rejects the reset because the OTP is not valid for the current email	Invalid
TC-FP-017	Step 2: Submit a new password shorter than 8 characters	New password	Ab1!	Password EC2	System rejects the password as too short	Invalid
TC-FP-018	Step 2: Submit password missing uppercase letter	New password	strongpass1!	Password EC3	System rejects the password because it lacks an uppercase character	Invalid
TC-FP-019	Step 2: Submit password missing lowercase letter	New password	STRONGPASS1!	Password EC4	System rejects the password because it lacks a lowercase character	Invalid
TC-FP-020	Step 2: Submit password missing digit	New password	StrongPass!	Password EC5	System rejects the password because it lacks a numeric character	Invalid
TC-FP-021	Step 2: Submit password	New password	StrongPass1	Password EC6	System rejects the password	Invalid

	missing special character				because it lacks a special character	
TC-FP-022	Step 2: Submit empty new password	New password	""	Password EC7	System rejects the submission and requires a password	Invalid
TC-FP-023	Step 2: Submit whitespace-only new password	New password	" "	Password EC7	System rejects the password as invalid	Invalid
TC-FP-024	Step 2: Submit null new password	New password	null	Password EC8	System rejects the password and shows a validation error	Invalid
TC-FP-025	Step 2: Submit empty confirm password	Confirm new password	""	Confirm EC2	System rejects the submission and asks the user to confirm the password	Invalid
TC-FP-026	Step 2: Submit confirm password that does not match the new password	Confirm new password	New password: StrongPass1!; Confirm: StrongPass2!	Confirm EC3	System rejects the reset and reports that the password values do not match	Invalid
TC-FP-027	Step 2: Submit non-string confirm password	Confirm new password	123456	Confirm EC4	System rejects the input and prevents reset	Invalid
TC-FP-028	Step 2: Submit multiple invalid inputs together	OTP, New password, Confirm new password	OTP: 000000; New password: weak; Confirm: different	OTP EC2, Password EC2/EC3/EC6, Confirm EC3	System reports all relevant validation errors and does not reset the password	Invalid
TC-FP-029	Step 2: Submit form	OTP, New password,	OTP omitted;	OTP EC6	System blocks	Invalid

	with a missing required field	Confirm new password	New password provided; Confirm provided		submission and highlights the missing required OTP field	
TC-FP-030	Step 2: Submit password with special characters and matching confirmation	New password, Confirm new password	New: P@ssw0rd!; Confirm: P@ssw0rd!	Password EC1, Confirm EC1	System accepts the password and allows password reset	Valid

5. Boundary Value Analysis

Inputs with measurable boundaries

The measurable boundaries identified from the requirement are:

- OTP length: exactly 6 digits
- New password length: minimum 8 characters

The email address and confirm password do not have measurable numeric or ordered boundaries in the supplied requirements, so BVA is not applicable to them.

OTP

Boundary	Why it is a boundary	Why it should be tested
Just below minimum: 5 digits	One less than the required 6-digit length	Verifies that the system rejects values below the required length
Minimum: 6 digits	Exact required length	Confirms the smallest valid OTP is accepted
Just above minimum: 7 digits	One more than the required length	Verifies that the system rejects values above the allowed fixed length
Nominal value: 6 digits	Standard valid OTP value	Confirms the normal successful path
Just below maximum: 5 digits	Same as the lower-bound side because the valid range is fixed at 6	Confirms the lower edge of the fixed-length rule
Maximum: 6 digits	Exact maximum allowed because the rule is fixed-length	Confirms the upper edge of the fixed-length rule

Just above maximum: 7 digits	One more than the permitted length	Confirms that values beyond the upper bound are rejected

New password

Boundary	Why it is a boundary	Why it should be tested
Just below minimum: 7 characters	One character shorter than the minimum required length	Verifies that passwords below the minimum length are rejected
Minimum: 8 characters	Smallest accepted length by the stated rule	Confirms the minimum acceptable password length is accepted
Just above minimum: 9 characters	One character longer than the minimum	Confirms that slightly longer valid passwords are accepted
Nominal value: 8+ characters with all required classes	Typical valid strong password	Confirms normal successful behavior
Just below maximum: Not defined	No maximum length is provided	No upper-bound boundary can be derived
Maximum: Not defined	No maximum length is provided	No upper-bound boundary can be derived
Just above maximum: Not defined	No maximum length is provided	No upper-bound boundary can be derived

Email address

- No measurable numeric or length boundary is defined.
- Boundary Value Analysis is not applicable.

Confirm new password

- The requirement is equality-based rather than length-based.
- Boundary Value Analysis is not applicable.

6.Test cases for boundary value analysis

Test Case ID	Input(s)	Test Data	Boundary Tested	Expected Result	Test Type
BVA-01	OTP	12345	<6 <6	Reject OTP because it is shorter than the required 6-digit length	Lower Boundary
BVA-02	OTP	123456	=6	Accept OTP because it matches the required length	Lower Boundary

			=6		
BVA-03	OTP	1234567	>6 >6	Reject OTP because it exceeds the required 6-digit length	Upper Boundary
BVA-04	OTP	123456	Nominal	Accept OTP as a standard valid value	Nominal
BVA-05	OTP	12345	<6 <6	Reject OTP because it is below the fixed-length lower/upper boundary	Upper Boundary
BVA-06	OTP	123456	=6 =6	Accept OTP because it is the exact maximum allowed length	Upper Boundary
BVA-07	OTP	1234567	>6 >6	Reject OTP because it is beyond the maximum allowed length	Upper Boundary
BVA-08	New password	A1!bcde	<8 <8 characters	Reject password because it is shorter than the minimum required length	Lower Boundary
BVA-09	New password	A1!bcdef	=8 =8 characters	Accept password because it meets the minimum required length and strength rules	Lower Boundary
BVA-10	New password	A1!bcdefg	>8 >8 characters	Accept password because it is longer than the minimum and still satisfies strength rules	Lower Boundary
BVA-11	New password	StrongPass1!	Nominal	Accept password as a typical valid strong password	Nominal

II. Pool B: FR-10: Order state machine

1. Requirement Analysis

Functional requirements summary

FR-11 requires the user order history view to:

- Allow a user to see only their own orders.
- Display for each order: order ID, date placed, total amount, and current status.
- Show the status text translated into Vietnamese and styled with distinct colors.

User inputs

The relevant inputs are:

- User authentication / authorization context
- Ownership relationship between user and orders
- Order records retrieved for the user
- Order status values
- Order total amount values
- Order date values

Assumptions

- Authentication is required and the user identity is derived from a valid token/session.
- The endpoint likely uses `/api/orders/my-orders` and does not accept arbitrary order IDs from the user.
- Order data is supplied by the backend; the user is not directly entering order fields.
- Status translation and color display are UI presentation details rather than additional user inputs.

---

2. Input Domain Identification

Input name: User authentication context

- Data type: Auth token / session context
- Purpose: Verify identity and enforce access control
- Valid domain: Authenticated user with valid token/session
- Invalid domain: unauthenticated request
- Constraints: Must be present and valid; must identify a user
- Dependencies: Needed to determine owner access to orders
- Assumptions: The system derives the user identity from the auth context before querying orders

Input name: Order dataset / order records

- Data type: Collection/list of orders
- Purpose: Source data for the order history view
- Valid domain: A readable list of orders belonging to the user
- Invalid domain: Missing list, null dataset, malformed order collection
- Constraints: Must be retrievable and correctly structured
- Dependencies: Depends on database query and ownership filtering
- Assumptions: Order history is based on backend-provided order data

Input name: Order status

- Data type: String
- Purpose: Display current order state in the history view
- Valid domain: pending, confirmed, shipping, delivered, canceled
- Constraints: Must be a recognized status value

- Dependencies: Depends on each order record
- Assumptions: Order status values are stored consistently in the backend

Input name: Order total_amount

- Data type: Number/integer
- Purpose: Display the total monetary amount of each order
- Valid domain: Valid numeric amount
- Constraints: Must represent a valid amount
- Dependencies: Depends on each order record
- Assumptions: The backend stores valid monetary amounts for orders

Input name: Order date

- Data type: Date/time string
- Purpose: Display when the order was placed
- Valid domain: Valid date/time value
- Constraints: Must be displayable as a date
- Dependencies: Depends on each order record
- Assumptions: The backend provides a valid `created_at` or equivalent timestamp

---

3. Equivalence Class Partitioning

Input	EC ID	Equivalence class	Type	Why it exists
User authentication context	EC1	Authenticated user	Valid	Represents the normal authorized access path.
	EC2	Unauthenticated request	Invalid	Ensures unauthenticated users cannot view order history.
	EC3	Invalid/expired token	Invalid	Ensures invalid auth is rejected.
User ownership context	EC4	Requesting user owns orders	Valid	Only own orders should be returned.
Order dataset	EC6	Valid list of user orders	Valid	Normal case where the user has order history.
	EC7	Empty order list	Valid	User has no orders but view should still render safely.

	EC8	Missing/null order dataset	Invalid	Prevents system failure when data is unavailable.
	EC9	Malformed/non-list order dataset	Invalid	Ensures the UI/backend handles bad data structure.
Order status	EC10	Recognized status values	Valid	Represents valid order states for display.
	EC11	Unknown/unsupported status	Invalid	Ensures invalid status values are handled gracefully.
	EC12	Empty/null status	Invalid	Prevents missing status from breaking the view.
Order total_amount	EC13	Valid numeric amount	Valid	Represents a normal order total.
	EC14	Missing/null amount	Invalid	Prevents invalid monetary display.
	EC15	Non-numeric/malformed amount	Invalid	Ensures bad amount data is handled safely.
Order date	EC16	Valid date/timestamp	Valid	Represents normal order history display.
	EC17	Missing/null date	Invalid	Ensures date absence does not break display.
	EC18	Malformed date	Invalid	Prevents invalid date formatting/display errors.

III. Pool C: FR-12: Dashboard

1. Requirement Analysis

Functional requirements summary

FR-13 defines the admin dashboard metrics:

- The dashboard must display total revenue.
- Total revenue must include only orders whose status is delivered.
- The dashboard must also display total number of orders.

User inputs

The relevant inputs are:

- Order data from the system/database

- Each order’s status
- Each order’s total_amount
- Admin access context

Assumptions

Because the requirement is somewhat implicit, the following assumptions are used for domain analysis:

- The dashboard reads order records from persisted storage.
- The total number of orders counts all order records, regardless of status.
- Revenue is calculated only from orders where status equals delivered.
- Admin access is already governed by FR-12 and is assumed to be valid for dashboard access.

2. Input Domain Identification

1) Order dataset

- Input name: Order dataset
- Data type: Collection/list of order records
- Purpose: Source of data for both metrics
- Valid domain: A readable collection of order records
- Invalid domain: Missing.
- Constraints: Must be interpretable as a list of order records
- Dependencies: Depends on backend/database availability and data retrieval
- Assumptions: The dashboard loads data from the backend

2) Order status

- Input name: Order status
- Data type: String
- Purpose: Determines whether an order contributes to revenue
- Valid domain: delivered, pending, confirmed, shipping, canceled
- Constraints: Must be one of the defined status values
- Dependencies: Depends on each order record in the dataset
- Assumptions: Status is stored as a string in the orders table, and order status is always valid ensured by backend(data retrieved from backend is correct 100%)..

3) Order total_amount

- Input name: Order total_amount
- Data type: Number/integer
- Purpose: Used to compute revenue
- Valid domain: Numeric monetary values
- Constraints: Must be a valid numeric amount
- Dependencies: Depends on each order record in the dataset
- Assumptions: The amount is stored in the database and should be treated as monetary data, and total_amount is always Numeric monetary values as ensured by backend(data retrieved from backend is correct 100%).

4) Admin access context [redacted]

- Input name: Admin access context [redacted]
- Data type: Authentication/authorization context [redacted]
- Purpose: Confirms the user is authorized to view the dashboard [redacted]
- Valid domain: Authenticated admin user [redacted]
- Invalid domain: Non-admin, unauthenticated [redacted]
- Constraints: Must satisfy admin authorization [redacted]
- Dependencies: Depends on authentication and role data [redacted]
- Assumptions: FR-12 governs access to admin features [redacted]

3. Equivalence Class Partitioning [redacted]

Input: Order dataset [redacted]

- EC1: Non-empty collection of order records [redacted]
  - Valid
  - Reason: This is the normal case where the dashboard must compute metrics from existing orders.
- EC2: Missing dataset [redacted]
  - Invalid
  - Reason: The dashboard should not proceed with a missing data source.

Input: Order status [redacted]

- EC3: Status = delivered [redacted]
  - Valid
  - Reason: This is the only status that should contribute to revenue.
- EC4: Status = pending / confirmed / shipping / canceled [redacted]
  - Valid
  - Reason: These are valid order states, but they should not contribute to revenue for FR-13.

Input: Order total_amount [redacted]

- EC5: Numeric monetary value [redacted]
  - Valid
  - Reason: This is the normal case for revenue calculation.

Input: Admin access context [redacted]

- EC6: Authenticated admin user [redacted]
  - Valid
  - Reason: This is the intended access context for the admin dashboard.
- EC7: Non-admin / unauthenticated / missing role [redacted]
  - Invalid
  - Reason: These contexts should not allow dashboard access.

4. Test cases for domain testing

Test Case ID	Requirement	Input(s)	Test Data	Equivalence Class ID	Expected Result	Test Type
DT-FR13-01	Display dashboard metrics for normal admin view	Order dataset, order status, total_amount, admin access	Dataset contains 5 orders: one delivered with amount 6000000, and 4 orders of the rest of status; role = admin	EC1, EC3, EC4, EC5, EC6	Dashboard displays total revenue as 500000 and total order count as 3.	Valid
DT-FR13-02	Display dashboard with no orders	Order dataset, order status, total_amount, admin access	Dataset is empty; role = admin	EC2, EC6	Dashboard displays zero revenue and zero total order count.	Valid
DT-FR13-03	Include delivered orders in revenue	Order dataset, order status, total_amount	Dataset contains one order with status = delivered and amount = 300000	EC1, EC3, EC5	Revenue includes that order amount.	Valid
DT-FR13-04	Exclude non-delivered orders from revenue	Order dataset, order status, total_amount	Dataset contains one order with status = pending and amount = 300000	EC1, EC4, EC5	Revenue does not include that order amount, but total order count still includes it.	Valid
DT-FR13-5	Reject special-character role input	Admin access context	role != “admin"	EC7	System rejects the request as invalid authorization input.	Invalid

DT-FR13-6	Reject special-character role input	Admin access context	null	EC7	System rejects the request as invalid authorization input.	Invalid

5. Boundary Identification

Input name	Validation rules	Min value	Max value	Range constraints	Assumptions
Number of delivered orders	Must be non-negative	0	None defined	`>= 0`	Count derived from orders with `status='delivered'`
Number of pending, canceled,shipping orders	Must be non-negative	0	None defined	`>= 0`	Count derived from order records
total_amount per order	Must be non-negative	0	None defined	`>= 0`	Each order amount is assumed valid
Admin access context	Must be authenticated admin	N/A	N/A	N/A	Not subject to BVA

---

6. Boundary Value Analysis

Input	Just below minimum	Minimum	Just above minimum	Just below maximum	Maximum	Just above maximum	Why test?
Number of delivered orders	`-1`	`0`	`1`	N/A	N/A	N/A	Verifies that negative counts are rejected, zero delivered orders are

							handled correctly, and the first valid delivered order is counted accurately.
Number of pending, confirmed ,shipping, canceled orders	`-1`	`0`	`1`	N/A	N/A	N/A	Verifies that negative counts are rejected, zero orders are accepted, and the first valid order is counted correctly for each status.
total_amount per order	`-1`	`0`	`1`	N/A	N/A	N/A	Verifies that negative order amounts are rejected, zero-value orders are handled correctly (if allowed), and the smallest positive order amount is processed correctly.
Admin access context	N/A	N/A	N/A	N/A	N/A	N/A	BVA not applicable; this is categorical/authentication data.

7. Test cases for boundary value analysis

Test Case ID	**Input(s)**	**Test Data**	**Boundary Tested**	**Expected Result**
BVA-FR13-01	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders;	delivered=0, pending=0, confirmed=0, shipping=0, canceled=0,	Minimum for all count and amount inputs	Dashboard handles the empty-order scenario correctly: all counts = 0 and total revenue = 0.
BVA-FR13-02	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping	delivered=1,pending=2, confirmed=2, shipping=2, canceled=2, order amount=1	Just above minimum for delivered order count	Dashboard displays 9 delivered order and revenue = 1.

	orders; total_amount per order		and order amount	
BVA-FR13-03	Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders	pending=1, confirmed=1, shipping=1, canceled=1, Order amount = 1.	Just above minimum for each non-delivered status count	Dashboard counts one order in each non-delivered status correctly and total orders includes all 4, with revenue = 0.
BVA-FR13-04	Number of delivered orders; total_amount per order	delivered=1, order amount=0	Minimum order amount for a delivered order	Dashboard accepts a delivered order with amount 0 and reports revenue = 0.
BVA-FR13-05	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; total_amount per order	delivered=2, pending=1, confirmed=1, shipping=1, canceled=1, order amount=100000	Nominal values for counts and order amount	Dashboard handles normal-sized metric values correctly and displays expected totals.
BVA-FR13-06	Number of delivered orders; Number of pending orders; Number of confirmed orders; Number of shipping orders; Number of canceled orders; total_amount per order	delivered=1, pending=1, confirmed=1, shipping=1, canceled=1, order amount=1	Just above minimum for counts and amount across multiple statuses	Dashboard handles a mixed-status dataset with the smallest positive amount and displays correct totals.